

RFC: Parallel Page Buffer for VFD SWMR

An Initial Sketch Design

John Mainzer

VFD SWMR uses a page buffer to assemble pages of metadata that have been modified in the last tick. These pages are copied to the metadata file and indexed – allowing VFD SWMR readers to obtain a consistent and valid view of HDF5 file metadata as of the end of the tick. Further, if a page of metadata is modified on tick x , it must be written to file before tick $x + \text{max_lag}$, since at that point, the page will be dropped from the metadata file, and any interested reader must be able to find it in the HDF5 file proper.

At present, the HDF5 library does not support a page buffer in parallel builds. Given that any page of raw data can be written to by any rank, developing a parallel page buffer for raw data would present significant cache coherency issues.

In contrast, HDF5 requires that each rank see the same sequence of dirty metadata. This fact is used to maintain coherency between the metadata caches on each rank, and as shall be seen, can be used to maintain coherency between metadata only page buffers on each rank.

As the role that the page buffer plays in VFD SWMR is already well documented in “RFC: VFD SWMR” [1], this paper starts with a discussion of the technical issues raised by a metadata only page buffer in the parallel HDF5 build. It then discusses the somewhat contradictory requirements of this page buffer vs. those of a page buffer built to support VFD SWMR. Finally, it outlines several ways to address these issues.

1 Introduction2

2 Parallel Page Buffer Design3

3 Design Conflicts Between the Parallel Page Buffer and VFD SWMR4

 3.1 Page Size..... 4

 3.2 Tick vs. Sync Point 5

 3.3 To Flush or not to Flush..... 5

4 Squaring the Circle5

 4.1 Page Size..... 5

 4.2 Tick vs. Sync Point 7

 4.3 To Flush or not to Flush..... 8

5 VFD SWMR Reader and the Parallel Page Buffer9

1 Introduction

In addition to being a requirement for VFD SWMR, a parallel page buffer for the parallel HDF5 build has a number of potential advantages.

Perhaps the most obvious of these is its removal of the need for small metadata I/O requests. Since metadata is read and written in pages, and the page size is configurable, the parallel page buffer allows us to read and write metadata in pages of size equal to that of the underlying file system with the obvious performance advantages.

Note that this implies a large page size – 1 to 16 MiB on typical parallel file systems. While this is awkward for VFD SWMR, it has significant performance advantages for parallel HDF5. To see this, first observe that in many if not most cases, all metadata will fit into a parallel page buffer with only a few pages – thus avoiding metadata I/O during the body of the computation. Also, when opening a HDF5 file in a large computation, a parallel page buffer opens the possibility broadcasting the initial contents of the page buffer from rank zero – thus avoiding the overhead of all ranks simultaneously loading metadata pages from the file system.

Unfortunately, while this large page size is convenient for the parallel page buffer, it presents issues for VFD SWMR. To see this, observe that each time a page is dirtied during a tick, it must be written to the metadata file or be bundled into an updater file. Obviously, a 1 Mib page size will impose more overhead than the current 4 KiB default page size used in VFD SWMR.

In the next three sections, I outline the design details of the parallel page buffer, the incompatibilities between the parallel page buffer and VFD SWMR, and several ways of addressing these issues. Note that I do not select a preferred solution here, as the optimal solution will be hard to select without experimentation, and will likely vary from HPC machine to HPC machine.

2 Parallel Page Buffer Design

The primary issue in parallel page buffer design is maintaining the necessary consistency between the page buffers on the various ranks. Thus, this section concerns itself only with this issue, and not with the architecture of the page buffer. While there is no need to address this point now, I expect this structure to be quite conventional.

Parallel HDF5 requires all operations that modify metadata be collective. While this greatly simplifies the parallel metadata cache design by forcing metadata caches on all ranks to see the same sequence of dirty metadata, messages from the past / future are still possible unless steps are taken to avoid them.

To see the problem, consider that even though all metadata caches on all ranks see the same sequence of dirty metadata, they need not see the same sequence of metadata accesses, which results in different LRU lists, and thus different sequences of entry flush and eviction. Combine this with the fact that ranks can drift out of sync, and it should be obvious that piece of metadata A can be read, modified and written to file by rank X before rank Y loads it.

The parallel HDF5 metadata cache solves this problem by disabling dirty metadata flushes between sync points, requiring the metadata caches on each rank to count bytes of dirty metadata generated, and enter a sync point every time this sum exceeds some user specified value. Slightly simplified, sync point processing in the HDF5 parallel metadata cache is follows:

- Between sync points, no dirty metadata is written to file.
- When the target number of bytes of dirty metadata is reached, enter a barrier. Upon exiting the barrier:
 - If rank zero, flush dirty entries from the local metadata cache as appropriate, construct a list of the flushed entries, broadcast this list to all other ranks, and then proceed.
 - If not rank zero, receive the list of flushed entries from rank zero, mark the listed entries as clean in the local metadata cache, and then proceed.

While this algorithm is a bit more restrictive than necessary, it is effective in maintaining the necessary coherency between the per rank metadata caches.

A similar algorithm can be used to maintain the needed parallel metadata page buffer consistency. Since the sequence of dirty metadata is not visible to the per rank page buffers, the metadata caches still count bytes of dirty metadata, and trigger a sync point whenever this sum exceeds some user specified value.

- Between sync points, while no pages of metadata may be written to file, the metadata caches on top of the page buffers may write dirty metadata to the page buffers at will. The page buffers may load pages as necessary, but may not flush dirty pages.

- When the target number of bytes of dirty metadata is reached, enter a barrier. Upon exiting the barrier:
 - All metadata caches must flush all dirty entries to their local page buffers
 - If rank zero, the parallel page buffer flushes dirty pages as appropriate, broadcasts the list of flushed pages to all other ranks, and then proceeds.
 - If not rank zero, the parallel page buffer receives the list of flushed pages, marks the listed pages as clean, and then proceeds.

Note the “as appropriate” in the rank zero page buffer’s flush of dirty pages. This will likely translate to “not at all” in cases where all the HDF5 file metadata fits into a page or two – which is likely given the large page size.

3 Design Conflicts Between the Parallel Page Buffer and VFD SWMR

There are several basic conflicts between the Parallel Page Buffer and VFD SWMR:

- VFD SWMR wants a relatively small page size to minimize I/O requirements, while the Parallel Page Buffer wants to match the page size of the underlying parallel file system
- VFD SWMR ticks are time based, while the sync points used by the parallel page buffer and the parallel metadata cache are triggered by the observed sequence of dirty metadata generation.
- VFD SWMR requires that dirty pages in the page buffer be written to file if they are not modified for `max_lag` ticks, while the parallel page buffer prefers not to write dirty pages of metadata to file until file close unless the volume of dirty metadata forces the issue.

While all these issues are solvable, they force us to choose between a selection of compromises. The optimum choice is unknown at this point and will likely vary depending on the host system.

In the remainder of this section, I expand on each of these issues in turn. Potential solutions are discussed in the following section.

Finally, note that this section and the next refer only to the VFD SWMR writer end of things. The VFD SWMR reader has its own issues – which are discussed in a subsequent section.

3.1 Page Size

The page size issue is very simple – VFD SWMR wants to keep page size small to maximize the dirty metadata to page size ratio, while the parallel page buffer wants a page size equal to that of the underlying parallel file system so as to avoid the locking issues inherent in small writes to parallel file systems.

In a nutshell, the two subsystems need to push page size in opposite directions to maximize efficiency.

3.2 Tick vs. Sync Point

A tick in serial VFD SWMR applications is usually in the order of several hundred milliseconds. In contrast, sync points may be frequent early in a computation as the necessary HDF5 data structures are defined, and then non-existent for the rest of the computation. Alternatively, if the recently developed support for filtered chunked datasets in parallel HDF5 is employed, we may have a sync point or two at every checkpoint write.

At first glance, this shouldn't be a problem. Only one rank (probably rank zero) will perform the functions of the VFD SWMR writer, and there is nothing to stop it from running end of tick processing on a regular basis. However, if a sync point triggers the write of a page before VFD SWMR is ready, it risks setting up a message from the future bug on the VFD SWMR reader.

To see how this can occur, suppose a page of metadata has not been modified prior to tick n . Thus, any reader accessing this page prior to tick n will read it directly from the HDF5 file. Now suppose that this page is modified. If it is written to disk before tick $n + \text{max_lag}$, it is possible that some reader which thinks it is still tick $n-1$ will read the page and receive a message from the future.

3.3 To Flush or not to Flush

VFD SWMR wants to flush pages that have not been modified for more than max_lag ticks to file, so that it can drop the page in question from the metadata file.

In contrast, the parallel page buffer can't flush pages between sync points¹, and prefers not to flush pages at all.

4 Squaring the Circle

Having outlined the conflicts between the parallel page buffer and VFD SWMR in the previous section, in this section, I outline possible solutions and discuss tradeoffs.

4.1 Page Size

For performance reasons, the parallel page buffer pretty much has to use a page size that is congruent with the underlying parallel file system. Thus, we are left with three options:

1. Have VFD SWMR use the same page size as the parallel page buffer.
2. Employ two page buffers, one stacked on top of the other with different page sizes.

¹ Not quite true – in theory, both the rank zero parallel metadata cache and the rank zero parallel page buffer can flush entries to file between sync points without danger if they have been resident since at least the last sync point. This has never been tested, and more importantly, offers a sufficiently narrow window, that it doesn't offer a general solution to the issues raised by VFD SWMR.

3. Develop a page buffer that supports two different page sizes on the same underlying file – a smaller one for VFD SWMR purposes, and a larger one for parallel page buffer purposes.

Option 1) would greatly increase the size of VFD SWMR metadata files / updater files. At first glance this would seem to greatly increase the overhead of VFD SWMR, since these large pages would have to be written out even when only small sections had changed. Given that the parallel page buffer uses these large pages for improved I/O bandwidth, this might not be as bad as it seems. Further, since we will be using a parallel file system, the increase in metadata file / updater file size may not be noticeable.

At least from the perspective of the VFD SWMR writer, this issue could be mitigated by changing the format of updater files to contain diffs between pages over ticks. This would greatly reduce the bandwidth required by the VFD SWMR writer to write the updater files. However, this does little to reduce the increased overhead for the auxiliary process and the VFD SWMR readers.

Option 2) is another way to square the circle. The basic idea is to build two different page buffers – one optimized for VFD SWMR, and one optimized as a parallel page buffer. Under the more likely case in which we are running either serial with VFD SWMR or parallel without VFD SWMR, we would just use the appropriate page buffer.

However, for the VFD SWMR in parallel case, we would stack the two page buffers – VFD SWMR page buffer on top of the parallel page buffer – and add in the necessary logic to make the VFD SWMR page buffer play well with sync points, and the parallel page buffer play well with end of tick processing. Note that this assumes that:

1. The free space manager uses paged allocation with page size matching that used by the VFD SWMR page buffer.
2. The parallel page buffer uses a page size that is some multiple of page size used by the VFD SWMR page buffer, and that these pages are aligned such that a parallel page contains no partial VFD SWMR page buffer pages.
3. The free space manager on all ranks is modified to ensure that metadata and raw data VFD SWMR size pages are not mixed in parallel page buffer pages.

Here, the VFD SWMR page buffer would behave as per serial VFD SWMR – requiring that the metadata cache be flushed into it each tick, and retaining all pages modified in any given tick for use in the construction of the metadata file / updater files.

On sync point, the metadata cache would be flushed into the VFD SWMR page buffer, and the VFD SWMR page buffer would be flushed into parallel page buffer. Note that the issue of potential messages from the future caused by the premature flush of VFD SWMR page buffer is left to a later section.

This approach has the advantage and disadvantage of building and maintaining two different page buffers optimized for their respective functions.

The advantage is that we can avoid the overhead of both when only one of them is needed – a point that agrees with our usual design philosophy of trying to avoid making computations pay for functionality that they don't use.

The disadvantage is having to build and maintain two different page buffers.

There is also the cost of a bit more complexity in the free space managers necessary for correct behavior when operating together.

Finally, two buffers imply increased memory footprint when VFD SWMR in parallel is enabled. Whether the reduced metadata file / updater file size will make this worthwhile is unknown at this point.

At first glance, option 3) looks impractical. However, if we require that VFD SWMR page size be an even divisor of parallel page buffer page size, and that VFD SWMR pages be aligned such that no parallel page buffer page contains partial VFD SWMR pages, this may not be as ridiculous as it seems at first glance.

The basic idea here is to maintain the necessary data on each VFD SWMR page in each parallel page buffer page in the parallel page buffer. This could be done many ways, but for now presume an array of structures, one for each VFD SWMR page in the host parallel page buffer page. Maintaining this per VFD SWMR page data shouldn't involve much overhead, and of course it can be skipped in the non VFD SWMR case.

With this array, it would be relatively easy to pick out the dirty VFD SWMR pages at the end of each tick, copy them to the metadata file or an updater file, and then mark them as clean again. Note that in addition, the serial version of the page buffer also tracks

1. Whether the page was loaded from file
2. Whether this page is part of a multi-page metadata entry, and if so, what is its length and first page.
3. Whether any write of this page must be delayed, and if so until what tick.

As shall be seen, items 1 and 3 become irrelevant if we make the `max_lag` infinite. Item 2 is required for an optimization used in the serial version of VFD SWMR. It could be removed, if necessary, but the change would have to be ported to the serial case for ease of maintenance.

Note also, that to play nicely with the existing serial SWMR code, we would have to use paged allocation with a page size equal to the VFD SWMR page size. As per option 2 above, the free space managers would have to be modified to ensure that VFD SWMR sized pages were not mixed in parallel buffer sized pages.

While I think this option is doable, whether it is attractive may well depend on how we address the tick vs. sync point issue.

4.2 Tick vs. Sync Point

As discussed above, the basic issue here is the possibility of message from the future bugs. Here, consider a page that is modified only once in the computation, say at tick n . After the initial modification, the new version of the page is held in the VFD SWMR page buffer until tick $n + \text{max_}$

`lag`, and then written to file. This necessary, as otherwise, a reader whose current tick is less than `n` might attempt to read the page and see the modification that, from its perspective, hasn't happened yet.

This is fine for VFD SWMR only, but suppose a sync point happens between ticks `n` and `n + max_lag`. In this case the modified page may be written to file early – creating the potential message from the future bug.

The question is, how to make VFD SWMR and the parallel page buffer work together to avoid this issue. Two viable options come to mind:

1. Make `max_lag` infinite, thus avoiding any issues with regards to when VFD SWMR pages must be flushed.
2. Make `max_lag` variable. Instead of expressing `max_lag` as a fixed number of ticks in the past, tie it to a tick number, with all metadata file entries / updater files past that point being retained. Initialize this `max_lag` to zero.

After each sync point, examine the versions of all VFD SWMR pages currently on disk. If the value of `max_lag` can be safely increased without the risk of messages from the future, do so up to some user selected number of ticks after the current tick, and trim the metadata file / list of updater files accordingly.

Option 1) just avoids the problem by defining it out of existence – albeit at the cost of allowing the size of metadata file and possibly the sequence of updater files to grow without bound during the computation. Given the typical capacity of parallel file systems and the fact that the metadata file (and the updater files if present) may be safely deleted once the file is closed, this may or may not be a significant issue. As a side benefit, this adds crash recovery support if we generate updater files.

Option 2) allows us to trim the metadata file / set of updater files somewhat, albeit at the cost of maintaining the additional data on each VFD SWMR page in the single cache option above. The code to do the scan will require some thought, and thus the implementation cost may be significant.

Further, sync points do not occur on any regular schedule, and thus space savings will be widely variable. Whether this is worth the added implementation costs remains to be seen. Also observe that the size of the metadata file / set of updater files can be minimized by flushing all parallel page buffer pages at each sync point. Whether this will be a net win also remains to be seen.

4.3 To Flush or not to Flush

Recall that VFD SWMR needs to flush altered / new pages that haven't been modified for more than `max_lag` ticks so they can be dropped from the metadata file, while the parallel page buffer prefers not to flush pages unless forced to by limited page buffer space.

This interacts with the tick vs. sync point discussion above.

If we make `max_lag` infinite, VFD SWMR imposes no flushes on the parallel page buffer.

If, on the other hand, we go with the notion of a variable `max_lag` calculated after each sync point, we must consider allowing VFD SWMR to require the parallel page buffer to flush some or all pages at each sync point. If we get the metadata I/O performance benefits we hope out of bundling metadata

into underlying file system size pages, this probably isn't an issue. That said, we will not know until we try.

5 VFD SWMR Reader and the Parallel Page Buffer

So far, this sketch design has been directed purely at the parallel VFD SWMR writer. However, a parallel VFD SWMR reader has its own issues – when to check for the next tick and how to ensure that all ranks agree on the current tick.

In the serial case, this is done by checking the system clock, and checking for a new tick if more than a tick worth of time has passed. Since there is only one rank, there is no possibility of disagreements as to the current tick.

This approach will not work in the parallel case, since there is no guarantee that the clocks in the different nodes are in sync. Further, this time check is done on HDF5 API call entry, which may also vary between ranks. Worse, there is the possibility of dependencies between HDF5 API calls on different ranks which could lead to a deadlock.

Unfortunately, the only solution I have come up with, while workable, could be awkward in some cases.

This solution, which is discussed briefly in “RFC: VFD SWMR”, punts the problem to the user application. Here, the user application is required to trigger a collective “check for next tick” operation at convenient points during the computation. Once triggered, processing is as follows:

- Enter a barrier
- If rank zero, on exiting the barrier
 - Load the current tick from the metadata file, and broadcast this value to all other ranks
 - If the current tick from the metadata file is greater than the current tick in the application, load the index from the metadata file, broadcast it to all other ranks and then proceed. Otherwise, just proceed.
- If not rank zero, on exiting the barrier
 - Receive the current tick in the metadata file from rank zero.
 - If the current tick received from rank zero is greater than the current tick in the application, receive the new index from rank zero, and then proceed. Otherwise, just proceed.

In addition to forcing the user application to handle the problem, this solution puts no upper bound on the time between checks for the next tick – making the notion of an infinite `max_lag` discussed earlier more attractive.

Hopefully, a better solution to this problem with parallel VFD SWMR readers will present itself. However, for now, the above is the best I have.

Acknowledgement

This work is supported by the U.S. Department of Energy, Office of Science under award number DE-SC0025856 for Phase I SBIR project “Toward Robust HDF5”.

References

1. J. Mainzer, N. Fortner et al., RFC: VFD SWMR, May 19, 2022,
https://support.hdfgroup.org/releases/hdf5/documentation/rfc/VFD_SWMR RFC 220519.pdf

Revision History

<i>November 4, 2025:</i>	Version 0 circulated for comment within Lifeboat, LLC.
<i>May 26, 2026:</i>	Version 1 circulated for comment within Lifeboat, LLC.
<i>June 1, 2026:</i>	Version 2 circulated for comment.